PingRoom Final Implementation Review

MERN real time chat application

Prepared by Tijil Sharma
Database Systems project
Final demonstration 20 October 2026

Abstract

PingRoom implements persistent direct and group messaging with MongoDB, Express, React and Node.js. The database separates shared users, bounded conversation membership and unbounded message history into three collections. Transactions keep messages and cached previews consistent. Compound indexes support ordered history, while aggregation produces unread counts and database insights.

The final implementation includes authenticated CRUD operations, real time events, per-recipient receipts, text search, cursor pagination and attachment links. Local verification completed with 13 API and database tests and 4 browser tests passing. The production React build and formatting checks also passed. This report maps the earlier reviews to the executable implementation and states the remaining production limitations.

Final submission

Deliverable	Location
Application and source	github.com/TijilSharma/PingRoom
Final report	output/PingRoom_Final_Review.pdf and .docx
Pitch presentation	output/PingRoom_Final_Pitch.pptx
Code explanation and demo	docs/CODE_WALKTHROUGH.md and docs/DEMO.md
Reproducible verification	tests/api.test.js, tests/browser/chat.spec.js and GitHub Actions

Review basis

Review 1 defined the problem, functional requirements and MongoDB design. Review 2 documented the database implementation and a 138-document seed. This final review uses those documents as project context and validates claims against the current code and executed checks. Earlier design targets are not treated as measured results.

1 Requirements and final scope

Requirement	Final implementation and evidence
FR 1 Accounts	Register, login, profile update and password-confirmed deletion. Bcrypt hashing and JWT authentication.
FR 2 Direct conversations	User search and direct-chat creation. Sorted participant pair plus a partial unique index prevents duplicates, including concurrent requests.
FR 3 Group conversations	Named groups with 3 to 50 members. Creator alone can rename or delete the group.
FR 4 Real time messaging	HTTP validates and persists messages. Socket.IO publishes committed message events to authenticated participant rooms.
FR 5 Message status	Sender begins in receipt arrays. Delivered/read require all participants. Repeated receipts are idempotent and cannot regress read status.
FR 6 Presence and typing	Per-user socket counts support multiple tabs. Typing events are transient, throttled and membership-checked.
FR 7 Message history	Newest-first query with a timestamp and ObjectId cursor. The client shows each page chronologically.
FR 8 Attachments	Validated HTTP(S) image/file links with display name. Binary upload and object storage are outside this implementation.
FR 9 Recent conversations	Conversation list sorted by updatedAt with cached lastMessage and aggregated unread counts.

Nonfunctional results

The responsive browser interface passed a 390 × 844 viewport test. Authorization and transaction behavior have automated coverage. The original sub-300 ms latency objective has not been load-tested. A single MongoDB replica-set member supports transactions but offers no node-failure tolerance. Horizontal scaling and sharding remain future work.

Final changes after Review 2

The development proxy now follows the configured API port. Database Studio no longer marks a hidden conversation as read. Dialogs support focus trapping and Escape. React supporting components have separate files, the source has consistent formatting, the test suite covers additional authorization and concurrency cases, and GitHub Actions runs the verification workflow.

2 Collections relationships and indexes

Collection	Document shape and rationale
users	username and email have unique indexes. passwordHash is excluded by default. avatarUrl, status and lastSeen hold profile/presence information. Timestamps record creation and updates.
conversations	type, title, ownerId, participants[], directKey and lastMessage. Participant ObjectIds form a bounded embedded membership array. Shared users remain referenced documents.
messages	conversationId, senderId, text, optional attachment, deliveredTo[], readBy[], status and editedAt. Each message has timestamps and belongs to one conversation.

Conceptual relationships

A user participates in many conversations, and a conversation has many users. The conceptual junction entity becomes the participants array. A conversation has many messages, and each message references one sender. Mongoose refs enable population but do not act as database-enforced foreign keys. Routes and transactions enforce membership and cleanup.

Index definitions

Collection	Index	Query or constraint
users	username: 1; email: 1	Two unique indexes protect account identity.
conversations	participants: 1, updatedAt: -1	Multikey compound index for membership-filtered recent conversations.
conversations	directKey: 1	Unique partial index where type is direct.
messages	conversationId: 1, createdAt: -1, _id: -1	Ordered history and stable cursor tie-breaking.
messages	text: text	MongoDB text search scoped by conversation.

Embedding and denormalization

Messages remain separate because history grows without a fixed bound and a BSON document has a 16 MB size limit. A small lastMessage snapshot avoids fetching message documents for each conversation preview. This repeats data deliberately; transactions maintain its consistency. The message text index is separate from the history index, so search has a different access pattern.

3 Application architecture and CRUD

Runtime structure

One Node process serves the production React assets, Express API and Socket.IO transport. MongoDB runs in Docker on loopback port 27018 with replica set rs0. The browser communicates with the application on port 3100. During development, Vite proxies both API and socket traffic to the configured server port.

Layer	Code entry point	Responsibility
React	client/src/main.jsx	Chat state, fetch requests, socket listeners and UI updates.
Components	client/src/components/	Authentication form, attachment form and Database Studio.
HTTP API	server/app.js	Validation, authentication, membership, CRUD and aggregation.
Realtime	server/realtime.js	JWT handshake, user rooms, typing and presence.
Persistence	server/models.js	Schemas, references, indexes and database connection.
Startup	server/index.js	Connect, reset stale presence, serve assets and handle shutdown.

CRUD coverage

Entity	Create and read	Update and delete
User	Register; login; current account; user search	Update username; password-confirmed account deletion.
Conversation	Create direct/group; list by membership	Owner renames groups; creator deletes conversation and messages.
Message	Send text/link; history and text search	Author edits/deletes; members acknowledge delivery/read.

Authorization boundaries

JWT middleware resolves the current account. Conversation membership is checked independently for history, writes, typing and receipts. Owner checks protect conversation management; sender checks protect message edits and deletes. Inaccessible conversations return 404. Database Studio limits activity aggregation to the user’s conversations but intentionally exposes global collection counts and index definitions for evaluation.

4 Transactions queries and consistency

Sending and changing messages

The send route validates content, then opens mongoose.connection.transaction. Within one session it checks membership, inserts the message and calls snapshot to update the conversation preview. It emits a Socket.IO event only after commit. Edits and deletes also recompute the newest preview within the transaction. If any operation fails, both collections retain their previous committed state.

Deletion behavior

Deleting a conversation removes its messages and the conversation in one transaction. Deleting an account removes its direct chats and owned groups. Other groups keep existing members, remove the deleted user’s messages and receipt references, recompute receipt status and refresh the cached preview. The account deletion endpoint requires the password again.

Stable history pagination

A cursor contains the last returned timestamp and message ID. The next page matches earlier timestamps, or the same timestamp with a lower ObjectId. The query sorts by createdAt descending and _id descending and fetches limit plus one rows. The extra row determines whether a next cursor is needed. This avoids omissions when messages share a timestamp. The default page contains 30 records, with a maximum of 100.

Aggregation examples

Result	Pipeline behavior
Unread counts	$match limits messages to member conversations, excludes the sender and already-read messages; $group counts by conversationId.
Conversation activity	$group counts messages and characters; $lookup resolves the title; $project selects fields; $sort orders volume.
Daily volume and statuses	$facet computes multiple summaries from the same filtered input. $dateToString groups dates by day.
Query-plan inspection	explain with executionStats reports the winning plan, keys/documents examined and returned records. The integration test verifies IXSCAN.

The rollback test deliberately throws after changing both collections and asserts that neither write survives. The pagination test inserts 65 messages with tied timestamps and asserts that traversal has no duplicates or missing records.

5 Real time behavior and security

Event lifecycle

Socket.IO verifies the handshake token and checks that the user still exists. The server joins the connection to a user-specific room. API handlers publish message events to participant rooms after persistence. Typing checks membership before notifying peers. Expiry timers disconnect sockets when their token expires.

Presence and receipts

An in-memory map counts connections for each user. Closing one tab keeps the user online while another tab remains. The final disconnect writes offline status and lastSeen. Receipt updates use set union to avoid duplicate IDs, then check whether the participant set is contained in readBy or deliveredTo. A read acknowledgement implies delivery. A visible chat can acknowledge reading; a hidden tab or the Database Studio view cannot.

Controls and boundaries

Control	Implementation and qualification
Password storage	Bcrypt with cost 12. Hashes are not returned in public account objects.
Request validation	Zod types, maximum lengths, ObjectId format and HTTP(S) attachment protocol checks.
Access control	Membership, ownership and sender checks on protected operations.
Abuse reduction	Helmet headers, 32 KB JSON limit, API/auth rate limits and typing throttling.
Session handling	One-day signed JWT in sessionStorage. Logout removes the local token, but there is no server-side revocation list.
Frontend rendering	React escapes text. Attachment links open with noopener and noreferrer. Links are external resources, not server-side uploads.

Production limitations

This version runs one application process and one MongoDB member. Socket events are transient and not replayed, while reconnect refresh recovers persisted history. It does not provide end-to-end encryption, password recovery, calls, file storage or multi-region deployment. Internet deployment needs HTTPS, removal of demo accounts, restricted database diagnostics, backups and operational monitoring.

6 Implemented user interface

The React interface combines a conversation sidebar, searchable history, editable messages and a message composer. Direct and group conversations use the same message model. The current desktop capture below comes from the running implementation.

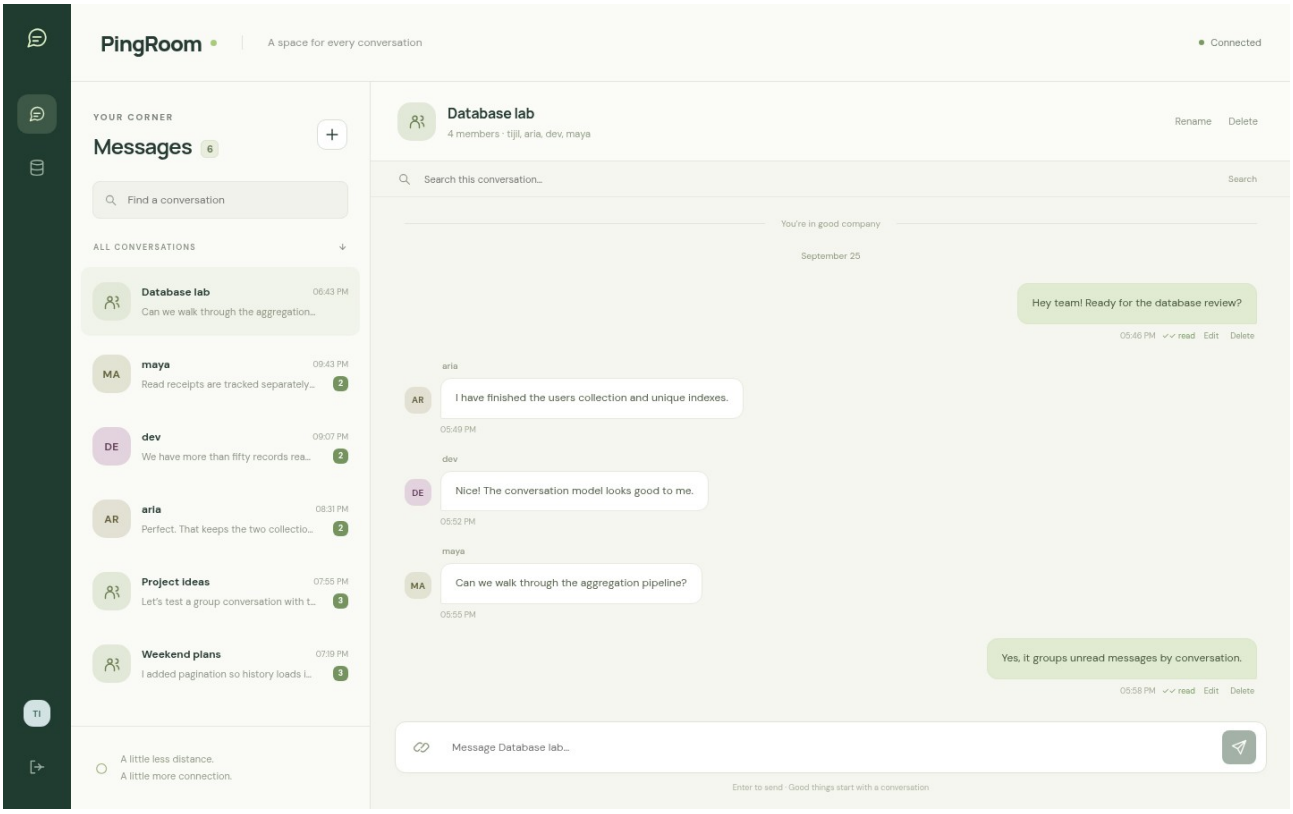


Figure 1 Conversation interface with membership, history, receipts and message actions.

Browser verification

Two isolated browser contexts log in as different demo users and exchange a live message. The test edits that message, checks the peer sees the update, searches the conversation, deletes the message and verifies its removal from the peer. Separate tests verify the mobile layout, keyboard dialog behavior and the hidden-conversation receipt regression.

Usability decisions

The UI displays connection state and request errors, prevents duplicate send clicks, ignores stale message-history responses after navigation, and expires typing indicators. Supporting React components separate authentication, attachment entry and database inspection from the chat screen. The source is formatted for direct code review.

7 Database evidence and sample data

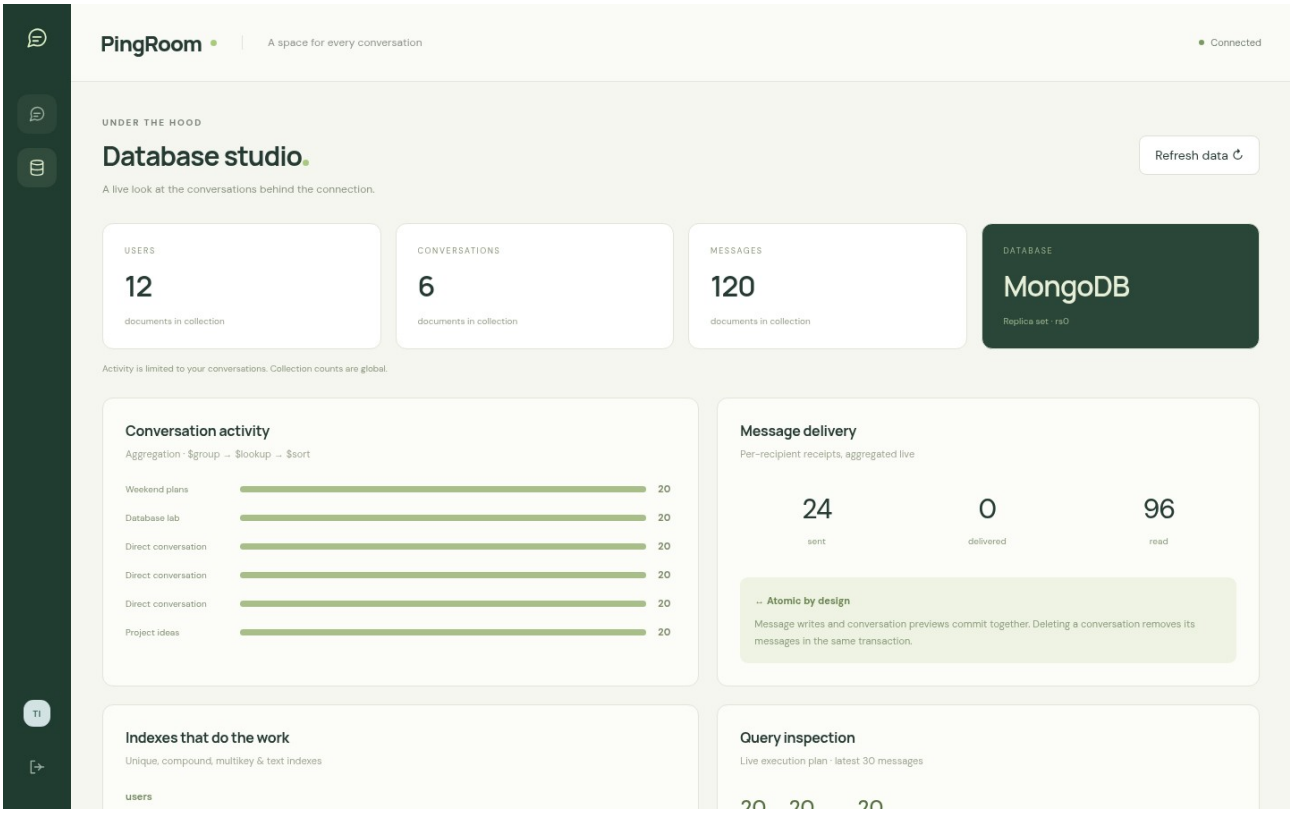


Figure 2 Database Studio showing live collection counts, activity summaries, indexes and query inspection.

Seed dataset

On an empty database, the transactional seed creates 12 synthetic users, 6 conversations and 120 messages: 138 documents. Three conversations are groups and three are direct chats, with 20 messages each. Re-running the seed preserves existing data. Live counts can change after demonstrations; the automated seed test checks the exact baseline in an isolated database.

Evidence interpretation

Database Studio presents membership-scoped activity alongside global counts. The winning query plan and execution statistics are generated from the running database. Index use is demonstrated for the selected history query; it is not a blanket performance guarantee. The one-member rs0 configuration demonstrates transaction support rather than high availability.

The seed has fewer messages per conversation than the default 30-message page. To demonstrate pagination in the UI, add at least 11 messages to a seeded conversation. The automated pagination test already exercises multiple pages with tied timestamps.

8 Validation deployment and conclusion

Check	Observed final result
npm run build	Production React bundle built successfully.
npm run format:check	All checked source files passed.
npm test	13 passing tests, 0 failures.
npm run test:ui	4 passing browser tests, 0 failures using system Chrome.
npm audit	0 known vulnerabilities after the compatible dependency update.

Test coverage

The API suite covers seeding, hashing, authentication, authorization, direct-chat de-duplication, message CRUD, receipts, stable pagination, group permissions, transaction rollback, scoped aggregation, index use, sockets, typing, multi-tab presence, account deletion, concurrent creation and input rejection. Browser tests cover the user-facing flows described on page 7. These tests establish functional evidence; they do not replace load, failover or penetration testing.

Reproduction

Install Node.js 22.12 or later and Docker Compose. Run npm ci, npm run setup, then npm start. Open http://localhost:3100. Setup creates a random local JWT secret, starts MongoDB, seeds only an empty database and builds React. For two-user demonstrations, use tijil@vitstudent.ac.in and aria@vitstudent.ac.in in separate browser sessions with the synthetic demo password PingRoom@123.

Conclusion and next phase

PingRoom completes the reviewed local text-chat scope with a working React application and demonstrable MongoDB design. The strongest database evidence is the combination of unique and compound indexes, membership-scoped aggregation, stable history cursors and transactionally maintained previews. A future production phase would add a multi-member replica set, shared presence infrastructure, operational monitoring, load benchmarks and managed file storage.

Source and evidence register

Project sources: server/models.js, server/app.js, server/realtime.js, client/src/main.jsx, scripts/seed.js and tests/. Earlier submissions: PingRoom Review 1 MERN ChatApp and PingRoom Review 2 Database Implementation, supplied PDFs. Final verification and screenshots were produced from the local implementation on 19 October 2026. Repository: https://github.com/TijilSharma/PingRoom.